

Development Prompts

This prototype was built end-to-end in a single Claude (Anthropic) session, working from the uploaded Product Requirements Document. The prompts below are the actual ones materially used, organized by area. Follow-up prompts that only said "continue" (to resume after the assistant's response ended) are omitted for brevity — they didn't add new instructions.

Understanding

Help me understand this project I want to first understand this so that I can build it further also help me draft a Document with all the main features , project overview, architecture , tech stack with all the available options for everything (so that I can choose my own stack based on the requirements understanding)

Architecture / scoping

"build this complete project and give me the final zip if you want any confirmations, confirm with me do not assume anything"

Follow-up clarifying answers given before implementation began:

Tech stack: "if my usual stack goes well for this build then go with it, or else give me options i'll choose" → confirmed as React + Node/Express + MongoDB (the candidate's existing stack).

AI provider: "Groq (fast, free tier, matches your stack)"

Scope: "Full Must-Have list, all features, larger codebase"

These three answers drove every architectural decision documented in `docs/ARCHITECTURE.md` (stack choice, TF-IDF retrieval instead of an embeddings provider, in-process job queue instead of a separate broker).

Backend

Internal implementation instructions the assistant worked from (derived directly from PRD sections 5-14), applied while writing `backend/src/models`, `backend/src/services`, `backend/src/controllers`, `backend/src/routes`, and `backend/src/jobs`:

- Implement the full data model needed for Users, Spaces, Projects, Materials, Chunks, Concepts, Mastery, Conversations, Quizzes, Recommendations, Events, AIUsage, and Jobs, with `user` / `project` references on every Project-scoped collection so ownership can be enforced consistently.
- Implement the Material pipeline (upload -> queued -> processing ->

chunking -> concept extraction -> ready/failed) as an asynchronous background job, not inline on the upload request.

- Implement the AI Tutor so it only answers from retrieved Project material, explicitly flags insufficient evidence instead of guessing, returns citations with material title + page, and treats retrieved material as data rather than instructions (prompt-injection awareness).
- Implement adaptive quiz question selection driven by mastery level, confidence (amount of evidence), and consecutive mistakes -- explicitly not a simple "wrong -> easier, correct -> harder" ladder -- and grade open-ended answers with explanatory feedback, not just a score.
- Implement mastery as a confidence-weighted evidence update (not a blunt overwrite), with a derived improving/stable/attention trend for Growth Analysis.
- Implement recommendations as a background job triggered by quiz completion and by repeated-mistake detection, superseding prior active recommendations.
- Implement an in-process background job queue with retries, exponential backoff, and idempotency keys to prevent duplicate processing on retry.
- Log every AI call (feature, model, tokens, latency, estimated cost, success/failure) through one central wrapper for observability.
- Enforce Project-level data isolation via a single ownership middleware used on every Project/Space-scoped route, returning 404 (not 403) for resources that exist but aren't the caller's.

Frontend

- Build a React + Vite + Tailwind SPA covering the full learning loop: auth, Home dashboard ("where was I, how am I doing, what's next"), Spaces list/detail, Project dashboard, Material upload with status polling, Tutor chat with inline citations and a visibly distinct style for "insufficient evidence" answers, an adaptive Quiz flow (MCQ + open-ended, immediate feedback, completion summary), Growth (grouped improving/stable/attention), Project and Global Analytics, and an Admin Dashboard (overview, users, per-user drill-down, activity feed with filters, AI usage, background jobs).
- Keep components functional and consistent (shared `card` / `btn-primary` / `input` Tailwind classes) rather than over-designing, given prototype scope.

AI / prompt design

- For every AI call that the application parses, require `response_format: json_object` with an explicit schema described in the system prompt, rather than parsing free text -- applied to Tutor answers, quiz question generation, quiz grading, concept extraction, and recommendation generation.
- For the Tutor specifically: instruct the model to treat retrieved

material excerpts strictly as reference data, never as instructions, even if the excerpt contains text that looks like a command -- this is the prompt-injection mitigation referenced in PRD section 15.

- Give the Tutor one real tool (`get_concept_mastery`) following the "Structured Tool / Application Request -> Backend Validation -> Execute" pattern from PRD section 8, rather than open-ended function access.

Testing

- Write unit tests for the logic that doesn't require a live database or API key: TF-IDF cosine similarity scoring, adaptive concept/difficulty selection (explicitly testing that it is *not* a pure wrong-to-easier/correct-to-harder ladder), mastery trend computation, and material chunking.
- Write integration tests for auth and Project-level data isolation that require a real MongoDB connection, designed to self-skip (not fail) when no database is reachable, since the build environment itself has no path to provision one.
- Write a small AI evaluation harness (`backend/src/eval/runEval.js`) specifically targeting Tutor groundedness and unsupported-question handling with curated test cases, structural/rule-based rather than model-graded, so it's fast and free to re-run after prompt changes.

Documentation

- Produce the full submission documentation set required by PRD section 20: README with setup/run/deploy instructions, this prompts log, an architecture doc with a diagram and explained tradeoffs, an AI usage doc distinguishing build-time vs. product AI, an evaluation approach doc, a known-limitations doc, and a future-improvements doc -- written to accurately describe what was actually built and verified in this session, not aspirationally.